

Proof of Stake on Sequentia

Consensus specification

Abstract. Sequentia is a UTXO blockchain for issued assets that defers its deepest security decisions to Bitcoin. Every Sequentia block commits to a Bitcoin block, and the chain reorganises whenever Bitcoin does, so a Sequentia transaction is never more settled than the Bitcoin history it is anchored to. The point of this discipline is settlement: an asset on Sequentia can be exchanged for bitcoin on the Bitcoin chain in real time, without the confirmation depth or protective timelocks that cross-chain trades normally require, because any Bitcoin reorganisation that reverses the bitcoin leg of a trade also reverses the Sequentia leg, by consensus rule. Blocks are produced by proof of stake. A stake is an ordinary unspent output of Sequence, the staking token, and it never moves: a verifiable random function elects producers privately in proportion to stake, and a public committee of up to 250 members certifies each block with a single aggregate signature. Finality is layered: a certified block is final at once, a checkpoint buried in Bitcoin makes it final against long-range attack, and the anchor overrides both. The same machinery supports vesting schedules whose tokens stake while still non-transferable, delegation that never surrenders custody, and reward distribution for staking pools enforced by consensus. This document specifies the mechanism and states which guarantees consensus enforces and which depend on incentives and monitoring.

1. Introduction

Sequentia is a blockchain for issued assets. It uses the unspent-transaction-output model and Bitcoin Script, and it derives from Elements, the Bitcoin sidechain codebase, inheriting its transaction format, its asset issuance, and its script extensions, including Taproot and transaction introspection. Elements assigns block production to a fixed federation; Sequentia assigns it by proof of stake. The signed-block machinery itself is unchanged: a block is valid only if it satisfies a challenge script fixed by consensus, and the rules in this document determine which keys that script names.

Three commitments shape the design.

Bitcoin's history prevails. Every Sequentia block commits to a Bitcoin block, and Sequentia reorganises whenever Bitcoin reorganises, immediately and unconditionally. No internal notion of finality can veto a reorganisation that Bitcoin has forced. Section 2 describes the mechanism, called anchoring, and the settlement guarantee it exists to provide; Section 6 ranks it above the protocol's two finality layers.

The supply of Sequence is fixed. Sequentia's staking token, named Sequence with ticker SEQ, has a supply of 400,000,000 units, all of it created in the genesis block. No coinbase ever creates more, and there is no block subsidy: a block producer's entire income is the transaction fees of its block.

Sequence is privileged only as stake. In every other respect Sequence is an asset like any other on the network. Transaction fees can be paid in any asset a producer accepts, and nothing requires holding Sequence in order to transact. Its one privileged role is staking: consensus accepts only Sequence as stake and checks the asset identifier explicitly.

Table 1 collects the protocol's constants. Each is a consensus rule, identical on every node. None is a setting: a node whose configuration tries to change one refuses to start.

constant	value	definition
supply of Sequence	400,000,000 SEQ	created in full at genesis; never increased
minimum stake	40,000 SEQ	0.01% of the supply; the smallest total stake that participates in block production (Section 3)
slot interval	30 seconds	the nominal block interval and the unit of the production time gate (Section 4.2)
maximum block weight	200,000	one twentieth of Bitcoin's; at 30-second blocks the chain adds capacity at the same rate as Bitcoin
unbonding period	43,200 blocks	15 days of slots; the minimum relative timelock of a staking output (Section 3)
committee cap	250	the certifying committee is the smaller of the cap and the number of registered stakers (Section 4.3)
quorum	$\text{floor}(K/2) + 1$, plus 1 at odd K	126 at the cap; computed from the actual committee size K
escape anchor gap	3 Bitcoin blocks	Bitcoin must advance this far before a stalled chain accepts a below-quorum block (Section 5.4)
checkpoint burial depth	2,016 Bitcoin blocks	about two weeks; the depth at which a checkpoint finalises (Section 6.3)
payout notice period	2,880 blocks	24 hours of slots; the minimum delay before a declared payout policy takes effect (Section 10)

Table 1. Consensus constants of the Sequentia network.

The document proceeds from the outside in. Section 2 covers the Bitcoin anchor and the settlement guarantee. Sections 3 to 6 cover the stake, the election, certification, and finality. Sections 7 to 10 cover the economic layer: rewards, vesting, delegation, and reward distribution. Section 11 states the trust model.

2. Bitcoin Anchoring and Real-Time Cross-Chain Settlement

2.1 The anchor

Every Sequentia block names a Bitcoin block, its *anchor*, by height and hash. Every Sequentia node watches the Bitcoin chain through a Bitcoin node, and three rules govern the reference.

Validity. The anchor must be a block of the Bitcoin chain as the validating node sees it.

Monotonicity. A block's anchor height is never lower than its parent's: the view of Bitcoin only moves forward along a Sequentia chain.

Canonicity. If a Bitcoin reorganisation removes an anchor from Bitcoin's best chain, every Sequentia block carrying that anchor is invalidated immediately, along with its descendants, and the chain reorganises onto its most recent validly anchored block. If a later Bitcoin reorganisation restores the anchor, the invalidated blocks are reconsidered and can rejoin the chain.

Canonicity applies continuously and at any depth: a Sequentia block is discarded the moment its anchor is orphaned, no matter what certification it carries (Section 6.4). Production keeps the chain close to Bitcoin's tip from the other direction as well: when committee members choose among competing proposals, they prefer the one with the most recent anchor (Section 5.1).

2.2 The settlement guarantee

The anchor exists so that bitcoin and Sequentia assets can be traded against each other in real time.

Consider selling an asset issued on Sequentia for a payment in bitcoin. Two Sequentia assets trade atomically in a single transaction; a trade against bitcoin cannot, because its two legs confirm on two chains that reorganise independently. The classical danger is that one leg is reversed after the other has settled: the bitcoin payment disappears in a reorganisation while the asset transfer stands, or the reverse. The usual defences are confirmation depth and contract timelocks sized for reorganisation risk, and they cost a cross-chain trade hours.

Anchoring replaces those margins with a consensus rule.

Property 1 (settlement inheritance). Let t_B be a Bitcoin transaction in Bitcoin block b , and let t_S be a Sequentia transaction in a Sequentia block s whose anchor is b or a descendant of b . In every future in which b leaves Bitcoin's best chain, s leaves Sequentia's best chain.

This is immediate from canonicity: whatever orphans b orphans every Bitcoin descendant of b , so the anchor of s is no longer canonical and s falls with everything built on it.

The consequence is that the two legs of the trade fail together. The seller treats the trade as settled once the block containing the asset transfer is quorum-certified, which takes seconds (Section 5), and its anchor is at or past b . From then on, any Bitcoin reorganisation deep enough to reverse the bitcoin payment reverses the asset transfer with it. The payment has to be in a Bitcoin block, but it does not have to be buried under more of them: burial adds nothing that Property 1 does not already give. The trade settles at the pace of one Bitcoin inclusion and one Sequentia certificate.

The same reasoning sizes the timelocks of hash-locked contracts. Payment channels and atomic swaps across independent chains normally pad their timelocks with a reorganisation margin on each side. Between Bitcoin and Sequentia the padding has no purpose, since a reorganisation that undoes one leg undoes the other. Timelocks cover the time honest parties need to act, and nothing else.

2.3 The anchor as a randomness beacon

The anchor has a second job: it seeds the leader election. The election seed of every height derives from the parent block's anchor (Section 4.1), so the randomness of the election comes from Bitcoin proof of work, which nobody inside the network can bias or grind.

3. Stake

Definition (staking output). A staking output is an unspent output that meets three conditions: its value and asset are explicit, and the asset is Sequence, since a hidden amount cannot carry publicly verifiable weight; its script is exactly the template below, stored bare so that validators recognise it the moment it is created; and its relative timelock, measured in wall-clock terms, is at least the unbonding period. The *weight* of a staking output is its value.

```
<csv> OP_CHECKSEQUENCEVERIFY OP_DROP  
  
    [ <bls_pubkey> OP_DROP <bls_pop> OP_DROP ]           committee registration  
(Section 4.3)  
  
    [ <liquid_locktime> OP_CHECKLOCKTIMEVERIFY OP_DROP ]   vesting lock (Section 8)  
  
<pubkey> OP_CHECKSIG
```

The staking script. Brackets mark fields an output may omit.

The staker set is a pure function of the unspent-output set: each node derives it by scanning that set at startup and keeps it current as blocks connect and disconnect. Because it has no state of its own, it survives reorganisations by construction. Weight adds up per public key across all of a key's staking outputs, and the 40,000 SEQ minimum applies to the total, delegated weight included (Section 9). Keys below the minimum take no part in the election and are left out of the totals against which other stakers are measured.

Two consequences of this definition carry most of what follows.

First, staking moves nothing. There is no transfer, no staking module, no custodian. Producing a block neither spends nor references the staking output (Section 4.2); the output confers weight simply by existing. Unbonding means spending the output, which the script's timelock delays by fifteen days, and the weight lapses with the spend.

Second, weight does not depend on spendability. An output that cannot currently be spent still carries its full weight. Section 8 builds vesting on exactly this.

4. Election

4.1 The seed

Every height H has an election seed

```
seed(H) = SHA256( anchor(parent) || H )
```

where $\text{anchor}(\text{parent})$ is the hash of the Bitcoin block the parent anchors. Every node computes the same seed on its own. Since the anchor is the output of Bitcoin proof of work, nobody chooses it and nobody can grind it, and the elections it seeds cannot be steered from inside the network.

4.2 Producer sortition

Who may produce is decided by private sortition. A staker with weight w , out of total eligible weight W , evaluates a verifiable random function over the seed with its secret key and reduces the output to a slot, uniformly distributed on $[0, W/w)$. It may publish a block from time $T_{\text{parent}} + \text{slot} \times 30$ seconds, where T_{parent} is the parent's timestamp. The block carries the VRF proof, and every validator checks the proof against the staker's key and the seed, and the timestamp against the slot. More stake means statistically earlier slots, so production frequency scales with stake.

The sortition is private: a VRF output can only be computed with the secret key, so until a block appears, nobody can tell who is entitled to publish it. There is no schedule to attack, bribe, or censor in advance. Absence costs nothing but time: if the earliest producer stays silent, the next slot opens thirty seconds later.

4.3 Committee selection

Certification needs the opposite of privacy: validators must know whose signatures a certificate requires. The committee is therefore selected publicly. For each seed, every eligible staker receives a ticket

```
ticket(pk) = SHA256( seed || pk ) / w
```

the hash read as a 256-bit integer and divided by the staker's weight. Stakers are sorted by ascending ticket, and the committee is the first K of them that have registered a committee key, where K is the smaller of 250 and the number of such stakers.

The committee key is a BLS12-381 key, registered by embedding it, with a proof of possession, inside a staking output. The fields have no effect on spending, and the registry learns them from the unspent-output set just as it learns weight. The proof of possession is checked when the block creating the output connects, and each staker may register one committee key.

A member certifies a block by signing its hash with its committee key. BLS signatures over the same message aggregate, so a certificate is a bitfield naming the signers within the ordered committee plus one aggregate signature: about three hundred bytes at any committee size, verified in a single check against the registered keys. The block itself is signed by its producer with the ordinary key of its

staking output, as the challenge script demands; only certification uses BLS.

The quorum at committee size K is

$$q(K) = \text{floor}(K/2) + 1, \text{ plus } 1 \text{ when } K \text{ is odd}$$

which is 126 at the cap. It is computed from the actual committee, so a network with sixty registered stakers runs a committee of sixty and a quorum of thirty-one.

Property 2 (quorum intersection). For every K , two quorums share at least $2q(K) - K \geq 2$ members, so certificates for two different blocks at one height require at least two members to have signed both.

The two constants answer different adversaries. The quorum is a bare majority, rather than the two-thirds of classical Byzantine fault tolerance, because the constraint that binds it is liveness: a two-thirds quorum is out of reach whenever a third of the committee is simply offline. The size is 250 because size is what resists capture: a coalition with a third of all stake gets a majority of a 100-member committee within days by luck alone, and a majority of a 250-member committee about once in decades. Size protects safety against a hostile third of the stake; the majority quorum protects liveness against an absent third of the committee.

Property 2 makes double certification attributable only if no member signs two proposals at one height. Honest members do not automatically satisfy that across round boundaries; Section 5.3 gives the rule that makes them.

5. Certification

5.1 Rounds

Time at a height is divided into rounds, measured from the timestamp of the first proposal. A round opens with a collection window in which members gather arriving proposals rather than endorsing the first one. When the window closes, every member backs the same proposal, chosen by a rule all nodes evaluate identically: the more recent anchor wins, and between equal anchors the lower producer VRF value wins. Normally all proposals carry the same anchor and the choice is simply lowest VRF; the anchor term matters only when a producer anchors stale, and it is the committee's half of the freshness guarantee of Section 2. Members back a proposal by flooding a BLS share over its hash, and any node that collects a quorum of shares assembles the certificate and floods it. If the round ends without a certificate, the next round begins at the same height with the next proposal in the ranking, so a silent producer costs one round.

The collection window and the round last $500 + 25K$ and $700 + 35K$ milliseconds, 6.75 and 9.45 seconds at the cap, growing with K because share traffic and share verification are the only per-member costs. They are liveness constants, identical on every node, and not validity rules: no block is accepted or rejected because of a clock. Certification runs at the speed of the fastest quorum, since one node with a quorum of shares completes the certificate; the slowest members add nothing to latency. A mistuned constant costs delay, never disagreement. The one case in which timing used to touch safety is removed by the share-lock (Section 5.3).

5.2 The certificate as an independent object

A certificate is self-verifying: hash, bitfield, and aggregate signature, checked against the registered committee keys. It travels as its own gossip object, separate from the block body, which can be hundreds of times larger.

The separation is for safety. If a partition swallows the block body while the certificate slips across, a node on the far side that holds the certificate knows the height is decided: it pins its finality floor to the certified block, requests the body, and neither produces nor countersigns a rival there, and the escape valve of Section 5.4 stays shut at that height. Without the certificate as an independent object, the far side of a partition could build on a height that had already been decided against it.

5.3 The share-lock

Rounds leave one honest path to conflicting certificates. Suppose block X collects its final share just before a round boundary. Some node assembles the certificate and starts flooding it, but flooding takes time, and the boundary arrives first. Members that have not yet seen the certificate treat the round as failed and back the next round's proposal at the same height. Their shares for X are still valid, so X's certificate completes anyway; if the second proposal also reaches quorum, there are now two certificates at one height, and every signature in both was honest. The members in the intersection signed twice, one round apart, exactly as their clocks told them to.

The share-lock closes this path. A member that has signed a share for X at height H signs no other block at H, and does not build above H on a chain that excludes X, until X can no longer certify. The lock releases once three things have happened: the round in which the member signed has ended; the member has asked its peers whether a certificate for X exists, a three-hundred-byte round trip; and one more round has passed without an answer. If a partition could be hiding the certificate, so that asking cannot settle the question, the member instead holds until Bitcoin has advanced three blocks past the parent's anchor, the same gap the escape valve uses.

Property 3 (certificate uniqueness). While the lock holds, a height can carry at most one certificate: for X to remain able to certify, at least q members must have signed it, all of them locked, so fewer than q remain free to sign a rival, and no rival reaches quorum.

In normal operation the lock binds nothing, because certificates complete quickly and release it at once; behind a failed proposal it costs one round. It is also what makes the timing constants of Section 5.1 harmless: with the lock in place, a race at a round boundary produces a late certificate instead of a second one.

5.4 The escape valve

A committee too depleted to reach quorum would stall the chain forever. A block certified by fewer than the quorum, at minimum by its producer alone, is therefore accepted under exactly one condition: its anchor is at least three Bitcoin blocks past its parent's, about thirty minutes of Bitcoin work. Liveness returns on Bitcoin's clock, and the quorum is never relaxed in normal operation. A below-quorum block never advances the finality point of Section 6.1, loses to any quorum-certified competitor, and is not

produced at a height where a certificate is known.

5.5 Ordering competing blocks

Competing blocks at a height are ordered by three keys: certification weight first, so a quorum-certified block beats everything else; then the producer's VRF value, lower winning; then the block hash, lower winning. The last key is globally deterministic, so nodes that have seen the same candidates order them the same way, and a symmetric tie between below-quorum siblings resolves identically everywhere without coordination.

Nothing in this ordering, and nothing anywhere else in the protocol, reorders a certified block. Certification is never revisited for height, arrival rate, or anchor freshness: a protocol that revisited it could be outrun by an adversary manufacturing certified blocks faster than the honest chain.

6. Finality

6.1 Immediate finality

The highest quorum-certified block on a node's active chain is its finality point. Nothing internal to Sequentia reorganises blocks at or below it; competing branches are rejected on arrival. The rule is affordable because of Properties 2 and 3: a second certificate at a finalised height cannot exist unless two members equivocated, and their own published signatures prove it.

Immediate finality protects whoever was online to see the certificate. It offers nothing to a node that was away, and nothing against histories fabricated later. Those are the subject of the next two subsections.

6.2 The long-range problem

Stake secures a chain only while it is bonded. Keys that held a majority of stake a year ago, and have since unbonded and sold, can sign an alternative history branching from any point of their tenure, at no cost and with valid signatures throughout. A node syncing from scratch sees two internally consistent histories and has no internal way to pick the real one. Every proof-of-stake system needs an external criterion. Sequentia's is Bitcoin.

6.3 Checkpoints

Anyone may commit a reference to a Sequentia block into Bitcoin: a transaction output carrying a fixed tag, the block hash, and the height. A node treats the referenced block as checkpoint-finalised once the block is on its own validated chain and the committing Bitcoin transaction is 2,016 Bitcoin blocks deep, about two weeks. Forks at or below that height are rejected outright.

Checkpoints are reject-only. A checkpoint never makes a node fetch, prefer, or adopt anything; it upgrades history the node has already validated from final-by-quorum to final-by-Bitcoin. A checkpoint that references a block the node has not validated does nothing, so forged or conflicting checkpoints are harmless. A brand-new node, which has validated nothing yet, ships with a static checkpoint, a

height and hash included in the software release, applied with the same reject-only meaning.

Property 4 (posterior-corruption closure). The unbonding period outlasts the burial window: fifteen days of slots against 2,016 Bitcoin blocks, about fourteen days in expectation. Stake that certified a history is therefore still bonded when the checkpoints over that history consolidate, so a coalition cannot unbond, sell, and then rewrite what it signed. Below a checkpoint, rewriting Sequentia means rewriting two weeks of Bitcoin proof of work.

6.4 Precedence

The three notions of settlement are strictly ordered: the anchor outranks checkpoints, and checkpoints outrank immediate finality. If Bitcoin orphans the anchor of a Sequentia block, that block is discarded, certified or not, checkpointed or not, and the finality point recomputes on the surviving chain. A node that drops a finalised block for this reason is following the protocol.

Property 1 forces this order. The promise made to a cross-chain counterparty is that a Sequentia settlement never outlives the Bitcoin history it is anchored to. A finality rule able to veto an anchor-driven reorganisation would break that promise exactly when it matters.

7. Producer Rewards

No block creates new Sequence, so a block's reward is its transaction fees, in whatever assets they were paid. Consensus fixes where they go: every coinbase output carrying value must pay the required payee of the height, a script $R(H)$ defined by the rules of Section 10, and a block that pays anyone else is invalid. For a producer with no declared distribution policy, $R(H)$ is a pay-to-witness-public-key-hash of its own key.

The producer builds its coinbase by evaluating the same function validators enforce, so the two cannot disagree. Rewards land in ordinary outputs, never inside a staking output: income from staking is spendable immediately, whatever locks bind the stake itself.

8. Vesting and Staking-Only Periods

Allocations of Sequence distributed at genesis vest on schedules with a cliff, a linear vesting period, and a *staking-only period*: an interval in which the tokens can stake but cannot yet be transferred. The staking-only period rests on the independence of weight from spendability established in Section 3.

A staking output may carry an absolute timelock, the `liquid_locktime` field of the template. Until that time the output cannot be spent, and so cannot be sold, transferred, or pledged. All the while it holds its full weight, produces blocks when elected, and earns fees, which are paid outside the output and are spendable at once. Non-transferability is obtained as non-spendability, with no covenant, no escrow, and no custodian.

The lock must be absolute. Relative timelocks are sixteen-bit quantities: at most 388 days in their time encoding, and 22.75 days in their height encoding at thirty-second blocks, so a staking-only period of

years cannot be expressed relatively. Absolute locktimes are thirty-two-bit and reach any date. They are written as wall-clock times, because block heights drift against the calendar over the years of a vesting schedule.

During the cliff an allocation must hold no weight at all, which is arranged by not yet creating a staking output: the allocation waits in a plain timelocked output, which confers nothing, and is converted into its staking output when the cliff ends. Where the conversion itself must be trustless rather than merely public, the pre-cliff output is a Taproot covenant that admits exactly one destination, the staking script it is due to become. That destination is a constant fixed when the allocation is created, so the covenant is a hash comparison, not a recursive construction.

9. Delegation

The key of a staking output has two jobs: it is the identity elected to produce blocks, which must stay online around the clock, and it is the key that spends the output. Kept in one key, the two jobs conflict. The key that controls the coins would sit on an exposed machine, and for a vesting-locked output that cannot be respent for years, the block-signing identity could never be changed. Delegation splits them.

A staker, the *controller*, assigns its production rights to a *signer* by funding a delegation record, a bare output of the form

```
<"SEQDEL"> OP_DROP <signer> OP_DROP <controller> OP_CHECKSIG
```

spendable by the controller alone. While the record is unspent, the controller's weight counts toward the signer in sortition, in committee selection, and against the minimum stake. The staking outputs themselves are untouched, and their key can stay offline. Resolution is one hop: weight delegated to a signer that has itself delegated does not follow the second edge, which rules out cycles. Consensus allows one unspent record per controller; a second would leave it ambiguous who may produce with that weight, and a block creating one is invalid.

Three properties follow.

Custody never moves. The signer appears nowhere in the staking output's spending condition, and cannot move a single token of the stake it produces with.

Exit is unilateral and immediate. The controller revokes by spending the record, with nobody's cooperation, no delay, and no movement of staked coins. Re-assignment spends the old record and creates the new one in a single transaction.

Assignment survives the vesting lock. The record is a separate output, so re-pointing the production rights of a locked stake never touches the stake. A compromised signer can be replaced at once, years before the coins can move.

Delegation is the protocol's staking-pool primitive. Holders below the minimum stake, or unwilling to run infrastructure, assign their weight to an operator. What the operator owes them is the subject of Section 10.

10. Reward Distribution Policies

Delegation lends weight, and under Section 7 the rewards would go entirely to the pool operator, leaving delegators to rely on the operator's bookkeeping. A producer can instead bind itself to a distribution policy, declared in a payout record, a bare output of the form

```
<"SEQPAY"> OP_DROP <activation> OP_DROP <mode> OP_DROP <parameter> OP_DROP <signer>  
OP_CHECKSIG
```

spendable by the signer alone. The policy in force at height h is the producer's record with the greatest activation not exceeding h ; the required payee $R(h)$ of Section 7 follows from it; and no signer may hold two unspent records with the same activation. There are two modes.

Direct. The parameter is a script, and $R(h)$ is that script: the coinbase must pay the committed destination, and the operator cannot quietly redirect a reward. The rule constrains the destination, not the split; the protocol does not judge whether the destination is fair to anyone. Direct mode is a public commitment with consensus force, not a trustless division.

Lottery. The coinbase must pay one participant, drawn from everyone whose weight counts toward the producer, the producer's own stake included, with probability proportional to weight. The draw is derived by hashing from the height's election seed, so every node computes the same payee and a block paying anyone else is invalid; and since the seed comes from Bitcoin, no participant can bias it. The parameter is a commission in basis points: with probability $c/10,000$ the reward goes to the producer outright. Lottery mode pays each participant exactly its proportional share in expectation, with no accounting machinery. What it does not do is smooth income: a participant with a share p of the pool receives the whole of a block's reward with probability p , not a p share of every reward.

Every policy is bound by the notice period: its activation must lie at least 2,880 blocks, twenty-four hours of slots, past the block that declares it, or that block is invalid. The notice is what turns inspection into protection. Exit is immediate and unilateral (Section 9), so a change that must stand in public for a day before taking effect cannot pay the operator anything before every delegator has had the chance to leave. Without it, an operator could take delegations under one policy and harvest the next block under another. Declaring a policy does not cancel the standing one: through the notice window both records exist, and the standing one governs.

11. The Trust Model

The guarantees above divide cleanly by who enforces them.

Enforced by consensus. A settlement anchored to Bitcoin history cannot outlive it, and history below a checkpoint cannot be rewritten without redoing two weeks of Bitcoin proof of work (Properties 1 and 4). A vesting lock cannot be circumvented, including by its own holder. A pool operator can never spend a delegator's coins, and can never block, delay, or condition a delegator's exit. A declared payout policy cannot take effect with less than a day's public notice. Under lottery distribution, expected income is exactly proportional to weight, and nobody can bias the draw.

Not enforced by consensus. Direct distribution does not make an operator fair; it makes the destination stable and public. The notice period protects the delegator who sees the notice; the protocol publishes the change and cannot make anyone look, so in practice its force depends on wallets and explorers surfacing pending changes to the delegators they affect. Lottery income is exact only on average, and a participant who needs smooth income must accept a direct-mode operator's terms or hold enough stake to produce alone. An operator with no stake of its own has no staking output in which to register a committee key: it can produce with borrowed weight, but to countersign it must bond stake of its own.

The design's position is that custody, exit, and commitment can be enforced, and that fairness beyond them is a market outcome: operators compete under policies they cannot change quietly, for delegators they cannot detain.

12. Summary

A stake on Sequentia is an unspent output of Sequence that never moves, and it confers the right to be elected in proportion to its value. Bitcoin provides the election's randomness through the anchor each block carries, and the anchor subordinates the whole chain to Bitcoin's history, which is what lets bitcoin and Sequentia assets trade against each other in real time. A block is produced by a privately elected staker, certified by one aggregate signature of a public committee, final on that certificate, and immovable beneath a Bitcoin checkpoint, always excepting the anchor, which outranks both. Weight does not depend on spendability, so tokens under vesting locks stake and earn while they cannot yet be sold. Production rights live in a revocable record separate from the stake, so delegation conveys no custody and exit is immediate. And a producer's reward destination is a consensus rule bound to its declared policy, so a pool's delegators hold a commitment their operator can neither break quietly nor change quickly.

Sequentia is developed by Concatena Labs. The network is named Sequentia; the token used for staking is named Sequence, with ticker SEQ.